

Learning Robot End-Effector Control for Planar Object Pushing

ME5418 Final Project Proposal

Taowen Liu, A0352191J

Xuanwei Liu, A0357433B

Chengtao Zheng, A0351467Y

Why:

- Pushing is a fundamental manipulation skill, useful when directly grasping an object is difficult or unnecessary, e.g., pre-positioning a component before grasping or assembly in industrial settings.
- Despite its simple appearance, pushing is contact-rich: the resulting object motion depends on contact location, pushing direction, object geometry, and friction, making a single hand-crafted controller hard to design for all configurations.
- In this project, we investigate whether reinforcement learning can learn a closed-loop Cartesian end-effector policy that uses a stick tool to move an object from a randomized initial pose to a desired target pose without relying on an accurate analytic model of the contact dynamics.

Conventional Algorithms:

- Conventional pushing approaches rely on geometric planning, model-based control, or trajectory optimization (e.g., quasi-static limit-surface pushing models or model-predictive control), using an estimated object pose and contact dynamics to plan a sequence of actions toward the target. However, accurate contact dynamics are difficult to model when geometry and friction vary, so these methods often require precise physical parameters; small modeling errors also compound over a push sequence, forcing frequent, computationally expensive replanning to stay on target.
- Instead, we believe reinforcement learning is a good fit for this problem: rather than relying on an explicit (and potentially inaccurate) contact model, an RL agent can learn a closed-loop feedback policy directly from experience, implicitly capturing how contact location and pushing direction affect the resulting motion. We favor RL specifically over supervised or imitation learning because it requires no dataset of expert demonstrations: the agent can instead discover an effective strategy on its own by directly optimizing task success through trial-and-error interaction, a natural fit for this sequential, closed-loop decision-making problem.

Problem Statement:

- The core task is PushT in the two-dimensional Gym-PushT environment: a robot end-effector must use a vertical stick to push a T-shaped object into a target region in a 512×512 workspace. Gym-PushT represents the stick's contact tip as a kinematic circle, which we interpret as a planar end-effector proxy. The core simulation therefore abstracts away the robot arm, its URDF, joint kinematics, and joint-level control.
- This is a single-agent robotic manipulation task at the Cartesian end-effector level. At each control step, the policy receives the planar position of the end-effector proxy and the object pose; no force/torque or tactile sensing is assumed. It commands the next planar target point of the end-effector proxy, which is moved by the environment's built-in position controller. The study therefore evaluates contact-point selection and closed-loop end-effector control rather than joint-space motion planning.
- At the start of each episode, the end-effector proxy position and the object's position and orientation are randomized. The target pose remains fixed in the core experiment, matching the standard Gym-PushT task.
- The policy must place the T-shaped object into the target region within a limited number of control steps (e.g., 300 per episode). Following Gym-PushT, an episode succeeds when object-target coverage exceeds 95%.
- Required robustness tests use broader held-out initial-pose ranges and multiple simulator damping settings.

Additional pushable geometries, such as rectangles or L-shaped objects, are optional generalization tests. Friction changes, randomized targets, and a full PandaStick robot model are further extensions because they require modifying or replacing the base environment.

RL Cast:

- **State space:** We use the normalized five-dimensional simulator state $s_t = [x_{ee,t}, y_{ee,t}, x_{o,t}, y_{o,t}, \theta_{o,t}]$, containing the current planar end-effector proxy position and T-object pose. The fixed target pose is known by the environment and is used to compute reward and evaluation metrics.
- **Action space:** The continuous action $a_t = [x_{ee,t}^{\text{cmd}}, y_{ee,t}^{\text{cmd}}] \in [0, 512]^2$ specifies the next Cartesian target of the end-effector proxy. Gym-PushT uses an internal PD controller to move the stick-tip proxy toward this point. A future PandaStick implementation can map the same planar command to the physical end-effector after inverse kinematics and reachability checks.
- **Reward structure:** We use a dense pose-progress reward with normalized position and orientation terms, so their relative weights are not determined by the choice of units:

$$r_t = w_p \frac{d_{\text{pos},t} - d_{\text{pos},t+1}}{L} + w_r \frac{d_{\text{rot},t} - d_{\text{rot},t+1}}{\pi} - c_{\text{step}} + R_s \mathbf{1}[\text{success}_{t+1}] - c_b \mathbf{1}[\text{invalid}_t].$$

Here $d_{\text{pos}} = \|p - p^*\|_2$ is measured in simulator units and $L = 512\sqrt{2}$ is the workspace diagonal. The orientation error is the shortest angular distance, $d_{\text{rot}} = |\text{atan2}(\sin(\theta - \theta^*), \cos(\theta - \theta^*))|$, measured in radians. Pose errors are evaluated after each action, and $\mathbf{1}[\cdot]$ denotes an indicator function. Fixed initial settings are $w_p = w_r = 1$, $c_{\text{step}} = 0.01$, $R_s = 10$, and $c_b = 1$. Reward tuning is limited to a small pilot study of the position-to-orientation weight ratio; the selected values are then held fixed across all methods, seeds, and test conditions.

Success is defined by the standard Gym-PushT criterion of more than 95% object–target coverage; the success bonus is awarded once and the episode then terminates. Episodes reaching the step limit are truncated without a success bonus. End-effector commands are clipped to the 512×512 workspace, with c_b applied to invalid commands. We do not penalize loss of contact, since moving around the object and establishing a new contact may be necessary to correct orientation. The progress terms supplement the environment’s coverage criterion with feedback before the object overlaps the target, while the per-step cost discourages unnecessary delays.

- **Neural Network Structure:** Since the observation space is low-dimensional, we plan to use a simple MLP (e.g., two hidden layers of 256 units, ReLU) for the policy and value networks, following common continuous-control PPO defaults (e.g., Stable-Baselines3), with the actor and critic sharing a trunk or fully separate if training is unstable.

RL Algorithms and Envisioned Results:

- We use Proximal Policy Optimization (PPO) as the sole algorithm in the core experiment, given its stability and wide use in continuous robotic control.
- We will train at least three independent random seeds and report the mean and standard deviation of task success rate, final target coverage, final position error, final orientation error, and steps-to-target over 100 held-out episodes per seed and test condition.
- Required tests cover nominal Gym-PushT episodes, broader held-out initial poses, and multiple simulator damping settings. These tests measure robustness without requiring a separate robot simulator.
- As a baseline, we will compare against a heuristic controller that first moves the end-effector proxy to a contact point behind the object (relative to the target direction) and then pushes it toward the target, re-planning as the object-to-target direction changes. We use this lightweight heuristic rather than a full model-based planner, since the latter would itself require an accurate contact/friction model – exactly the dependency RL avoids. We expect the learned policy to perform better on cases requiring multiple contacts or simultaneous position/orientation adjustment.